

GMS - Architecture & Implementation Design

1. Introduction

Gmail Management Service is a backend service designed to automate Gmail inbox management. It retrieves emails from the past 24 hours, analyzes those mails and categorizes them, and generates summaries.

The processed emails are combined into a **Single Email - Digest** which contains details such as the **Sender, Subject, and Summary**, along with action buttons such as **Delete, Archive, Mark as Important, Mark as Spam, and Reply**. The digest is automatically delivered to the user at a scheduled time.

2. Goal

The goal of **Gmail Management Service** is to automatically process emails received in the past 24 hours and provide the user with a **single daily email** containing **categorized emails, their summaries, and the required action buttons**.

3. Background / Current State

Email management is generally performed manually by users. Users need to review incoming emails, identify important messages, and decide how each email should be handled.

As the number of emails increases, it becomes difficult to quickly identify relevant messages and keep track of all emails received during the day. Emails may contain different types of information, including important messages, normal communication, promotional content, and spam.

GMS is designed to simplify this process by automatically processing emails received during the previous 24 hours. The service categorizes the emails, generates summaries, and combines the processed information into a **single daily email digest**. The digest also provides actions for managing individual emails.

4. Requirements

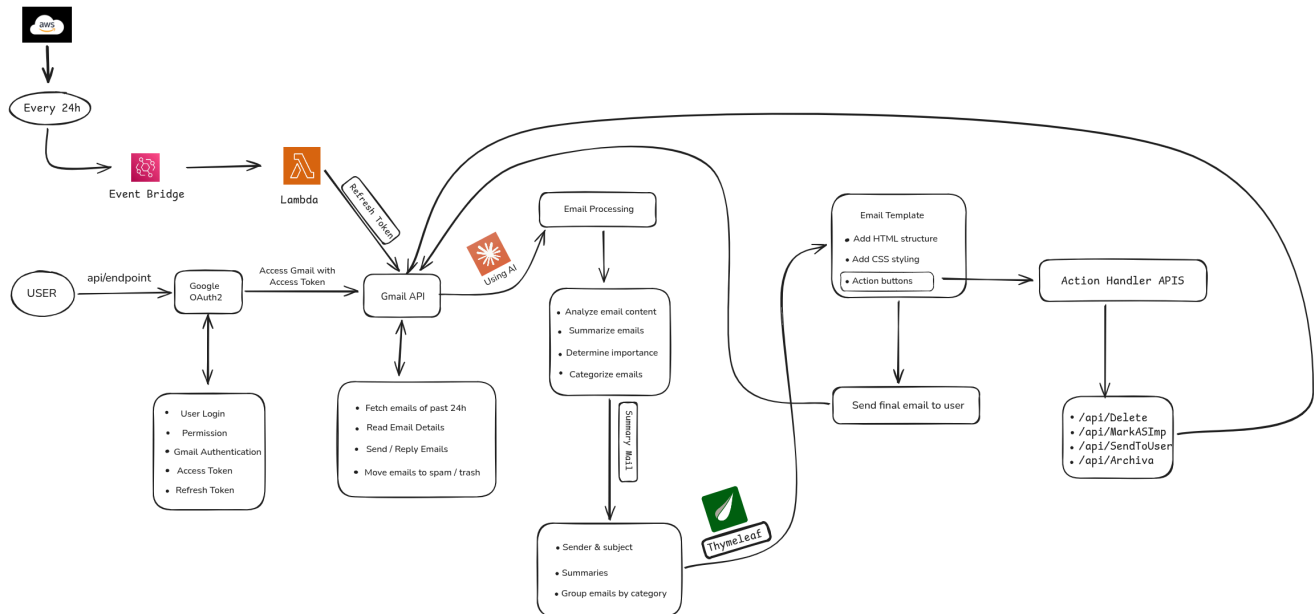
The Gmail Management Service should:

- Retrieve emails received within the **past 24 hours**.
- Analyze and categorize emails as **Important, Normal, Promotional, or Spam using AI**.
- Generate a **summary** for each processed email.
- Generate a **single daily email** containing the categorized emails and their summaries.
- Provide actions buttons such as **Delete, Archive, Mark as Important, and Reply**.

- Automatically deliver the digest to the user at a **scheduled time using AWS**.

5. High-Level Architecture

The Gmail Management service architecture consists of AWS services, Gmail APIs, AI-based email processing, and email generation components. The system is triggered every 24 hours to fetch and process emails received during the previous 24 hours.



The system is triggered every 24 hours by **AWS EventBridge**, which invokes **AWS Lambda** to access the Gmail API and retrieve emails received during the previous 24 hours. The user's Gmail account is accessed through **Google OAuth 2.0**, which provides the required authorization and access tokens.

The retrieved emails are processed using AI to **analyze their content, determine their importance, categorize them, and generate summaries**. The processed information is then passed to **Thymeleaf** to generate the daily email digest containing the sender, subject, summary, and action buttons. The **Action Handler APIs** handle the actions selected by the user, such as **Delete, Mark as Spam, Send to User, and Archive**, by performing the corresponding operations through the Gmail API.

6. Authentication & Authorization

GMS uses **Google OAuth 2.0** to obtain authorized access to the user's Gmail account. The user does not provide their Gmail password or a separate Gmail API. Instead, the user grants the required Gmail permissions to GMS, and Google provides OAuth tokens that the backend uses when calling the Gmail API. To access user mail information and perform repetitive tasks.

Step 1: User starts Gmail authentication

The user accesses the authentication endpoint:

```
GET /auth/google
```

Step 2: User signs in to Google

The user signs in with the Google account they want to connect to Gmail Management Service. The user then sees the permissions requested by Gmail Management Service and chooses whether to allow access. For Gmail Management Service, the required permissions are based on the operations the service performs. For example, Gmail provides scopes for reading messages, modifying messages, and sending mail. etc...

Step 3: Google redirects the user back to Callback URL

```
GET /auth/google/callback?code=AUTHORIZATION_CODE&state=...
```

The important value here is the **authorization code**. From the call back url we receive the Authorization code. The authorization code is temporary and is not itself used to read Gmail messages.

Step 4: Exchange Authorization Code for Access Token

After receiving the authorization code, GMS sends the code to Google's OAuth token endpoint:

```
POST https://oauth2.googleapis.com/token
```

The request contains the **authorization code**, **client ID**, **client secret**, **redirect URI**, and **grant type**.

Google validates the request and returns an **access token** and, when offline access is requested, a **refresh token**.

```
{
  "access_token": "ACCESS_TOKEN",
  "refresh_token": "REFRESH_TOKEN",
  "expires_in": 3599,
  "token_type": "Bearer"
}
```

- **Access Token** – Used by GMS to make authorized requests to the Gmail API.
- **Refresh Token** – Used to obtain a new access token when the current access token expires.

Step 5: Access Gmail API and Retrieve Emails from the Previous 24 Hours

After obtaining a valid **access token**, GMS uses it to make an authorized request to the Gmail API. The request is configured to retrieve only emails received during the **previous 24 hours**..

```
GET https://gmail.googleapis.com/gmail/v1/users/me/messages?
q=newer_than:1d/
Authorization: Bearer ACCESS_TOKEN
```

The `q` parameter is used to specify the Gmail search criteria for emails received during the required time period.

The Gmail API returns the message IDs that match the search criteria:

```
{
  "messages": [
    {
      "id": "18abc123",
      "threadId": "18abc123"
    },
    {
      "id": "18def456",
      "threadId": "18def456"
    }
  ]
}
```

Then we use the returned message IDs to retrieve the details of each email:

```
GET https://gmail.googleapis.com/gmail/v1/users/me/messages/{messageId}
Authorization: Bearer ACCESS_TOKEN
```